An AI agent that works on your code, on your machine.

DiveeOI is a self-hosted web app where AI agents read, write, run and remember inside your projects. The server and project data stay under your control; provider, web-search and remote MCP traffic goes where you configure it.

WHAT IT IS

A SolidJS web app in front of an Effect-TS server. You open a project directory and talk to an agent in a chat: it **explores your codebase**, writes and edits files, runs your tests and linters, searches the web when you ask it to, uses any MCP server you connect, and keeps a memory of your project between sessions. Targeted Learning can also index books and documents for grounded sessions, with separate library and student-memory stores.

WHAT IS NOT IT

Not a hosted IDE or model provider. You run the server and choose its authentication mode and model providers. Basic authentication and JWT account flows are supported; there is no hosted DiveeOI service required to use your own instance.

How it works

1. **Open a project.** The agent works in that project/worktree context, subject to the tool permissions and filesystem rules you configure.
2. **Write a sentence.** “Fix the flaky session test”, “add a migration for the new column”, “summarise my open pull requests”. No commands, no syntax.
3. **The agent works through tools** — read, grep, glob, edit, write, bash — each one behind a permission rule you set, and each one visible in the transcript as it happens.
4. **Review the work.** Tool actions and their results are visible in the transcript; use the diff and Git views to inspect the changes.
5. **Or let it run on its own.** A background task is a session with a schedule: it re-runs on a cron expression, with its own agent, its own model, and its own log.

THE MODEL IS YOURS

DiveeComposer is a router, not a model: it resolves every turn to the best free model that is actually reachable right now, and remembers which models answer. A model that fails is left alone for a day. Bring your own key for anything else — Anthropic, OpenAI, Google, OpenRouter, Ollama on your own box.

THE SHAPE OF IT

`bun run dev:both` starts the server and the app. The browser is just a client: the server owns the sessions, the tools, the permissions and the database. Behind a proxy, on your LAN, or over SSH — the same build.

The limits, stated plainly.

A self-hosted agent is a different bargain from a hosted one. Here is what you are actually getting, and where it stops.

THE MODEL

- **Free models are free models.** The bundled free tier is genuinely free and genuinely good enough to work with — and genuinely below a frontier model on hard reasoning. The app does not pretend otherwise.
- **The free tier moves.** Models are added and retired. The router re-resolves every turn, and a model that errors or answers with nothing is not used again for a day.
- **Reachability is not a given.** "Free" in the catalogue may still need a key. The router prefers what is actually reachable — which is why the bundled tier is the one it picks.
- **Your key, your bill.** Point it at your own provider and the cost, the rate limits and the data path are yours.

THE AGENT

- **It can be confidently wrong.** It writes code the way a good junior engineer writes code: fast, plausible, occasionally wrong. The diff is the safety net — read it.
- **It only knows what you give it.** It searches as it goes rather than holding an index of your whole repository. Ask it to look, and it looks.
- **Permissions are the real control.** Per tool, per pattern: "allow `git *`", "deny `rm *`", "ask on `external_directory`". A deny is a deny.
- **Background tasks are still agents.** A scheduled task runs unattended with the permissions it has. Give one the power to push, and it will push unattended.

THE PRODUCT, TODAY

- **Actively evolving.** DiveeOI is developed in the open; use the current release hand-off for build, test and deployment status rather than relying on an old feature snapshot.
- **Web only.** The terminal UI was removed on purpose. Everything happens in the browser.
- **Authentication is your responsibility.** Basic and JWT/account modes are supported. Keep authentication enabled, use strong secrets, and close registration after initial account setup.
- **No built-in TLS.** Terminate it at nginx or Caddy. Protect the core app database and the separate Targeted Learning data directory, not just the URL.

OPERATING IT

- **Persistence is split by subsystem.** Core sessions live in the application database; Targeted Learning keeps books/vectors in `books.db` and student memories in `student.db`. Back up each data directory.
- **A restart is safe.** Tasks that were running when the process died are picked back up: stale ones are marked interrupted, and a task's call journal is replayed instead of repeated.
- **Remote sessions run remotely.** Over SSH the agent works on the remote machine; `ctx_local` runs a single tool call back on your laptop.
- **Review every external boundary.** Prompts go to the model provider you select; web tools and remote MCP servers also make network requests. Self-hosting does not make those integrations offline.